

LeaFTL: A Learning-Based Flash Translation Layer for Solid-state Drives

2026. 08. 19

Presentation by Sung Jinwook, Jung Huichan
sungjw0408@dankook.ac.kr, a011214@dankook.ac.kr

Contents

1. Introduction
2. How to Create Learned Segments?
3. How to Mange Learned Segments?
4. Preserve Other FTL Funcutions
5. I/O operation
6. Evaluation
7. Conclusion

1. What is LeaFTL?

- Black-box Optimized FTL
- Using Simple Machine Learning at Address Mapping

1.1. Limitation of Page-level FTL

- Page-level FTL
 - Records LPA to PPA Entry per Each Page
 - Requires 8(=4+4) Bytes per Page → ex. 160 Bytes to Record 20 Pages
 - If Capacity of SSD Increases, Also Mapping Table Get Bigger
- Learned Index Segment of LeaFTL
 - Can Compact In-bound Mapping Entries into A Segment
 - Segment is Only 8-Byted Struct → ex. 20 Pages Can be Compacted to 8 Bytes!
 - | | | | |
|-----------|-----|-----|-----|
| 1 | 1 | 2 | 4 |
| S_{LPA} | L | K | I |

(Byte)

1	1	2	4
S_{LPA}	L	K	I

1.2. Limitation of Heuristics

- Human-driven Heuristics
 - Relies on Temporal/Spatial Locality (ex. DFTL)
 - **If Workload Breaks the Rules**, Cache Miss Occurs → Performance Degraded
- **Linear Machine Learning Model** of LeaFTL
 - Learns Mapping Distribution that is Allocated in Runtime
 - **Identifies Mapping Patterns** by Itself
 - Can **Apply Rules to New I/O Requests**

2. How to Create Learned Segments?

2.1. Learning Addr. Mapping Patterns

- Contiguous Access Pattern

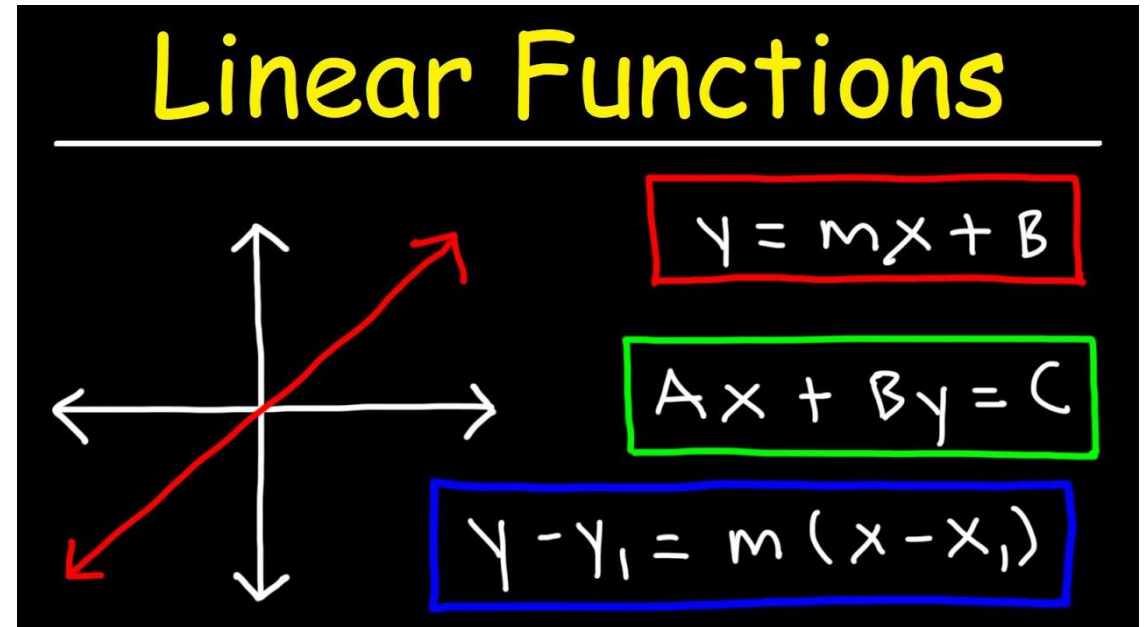
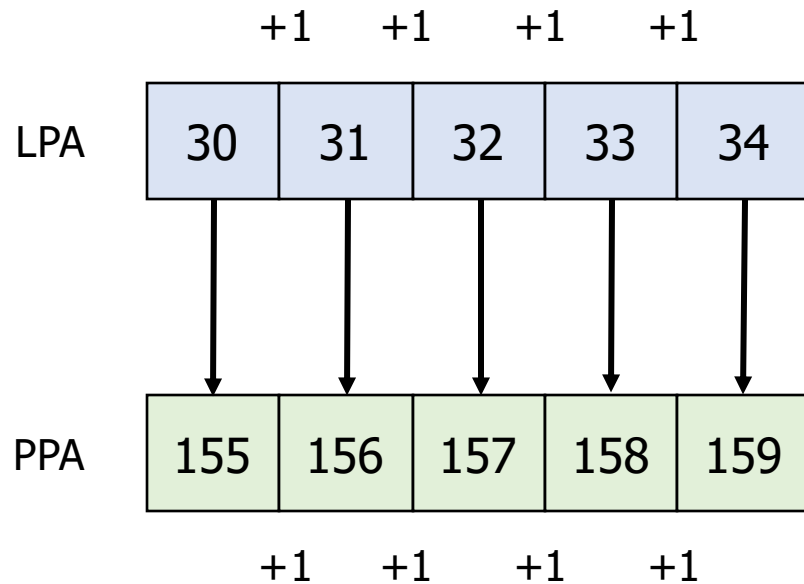
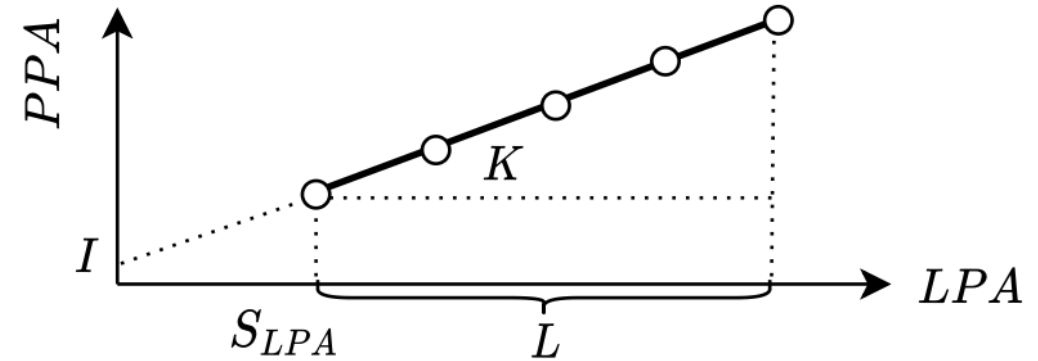
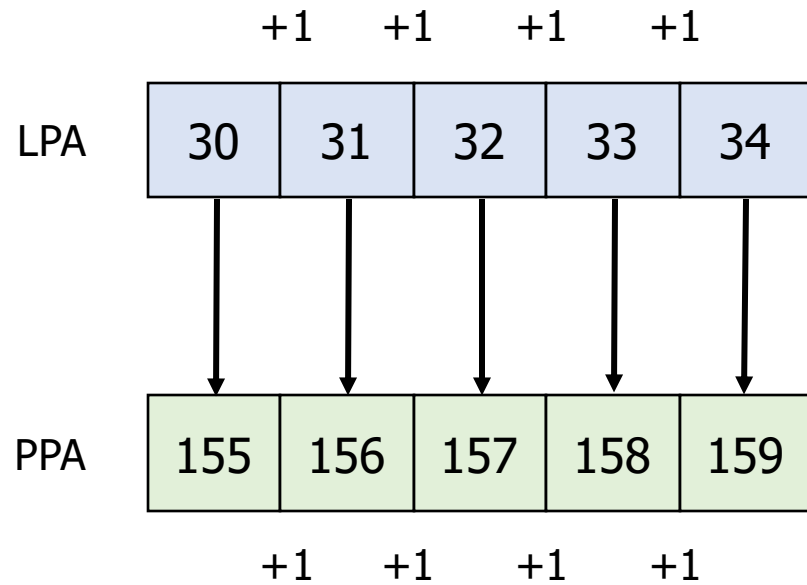


Image Ref: <https://www.youtube.com/watch?v=BtcKotD6Ni8>

2.1. Learning Addr. Mapping Patterns

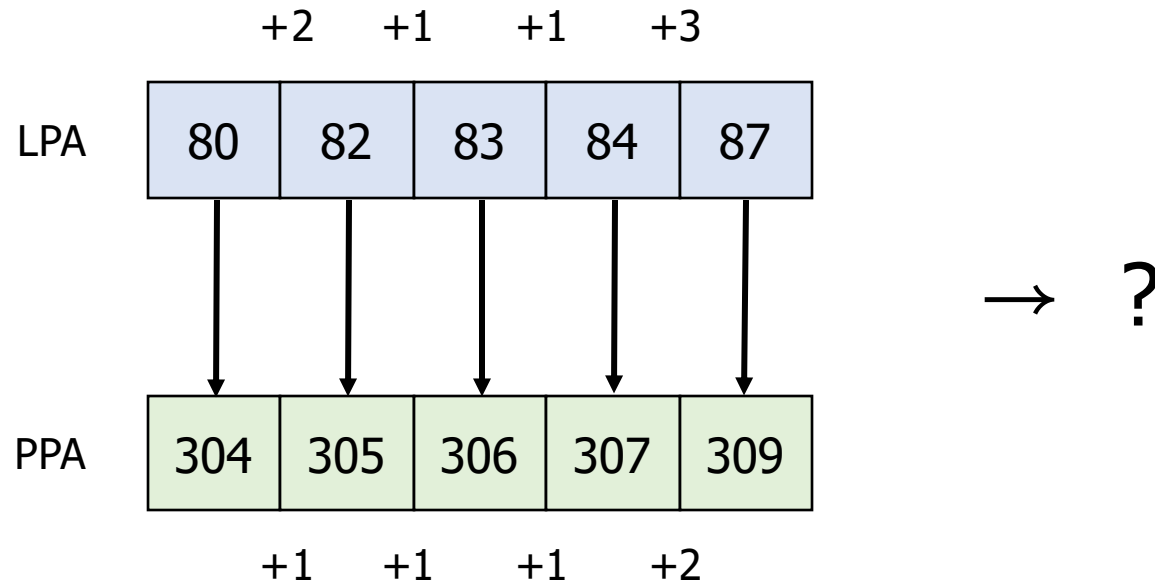
- **Linear Regression** of Machine Learning
 - Finding **A Linear Equation** that Best Defines Data Distribution
($y = Kx + I$)



(a) Precise Linear Approximation

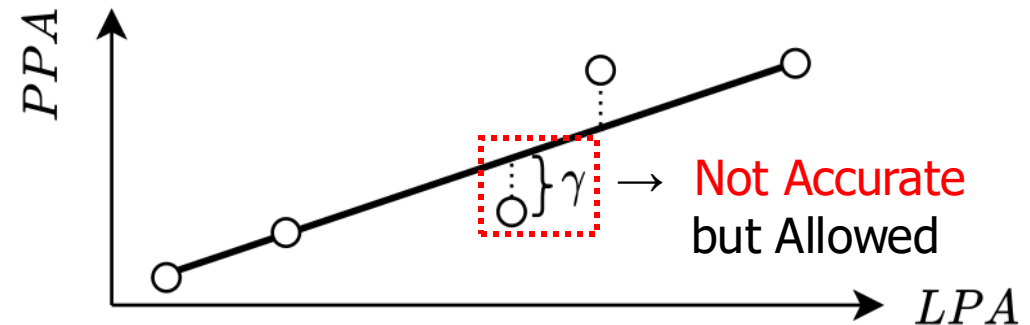
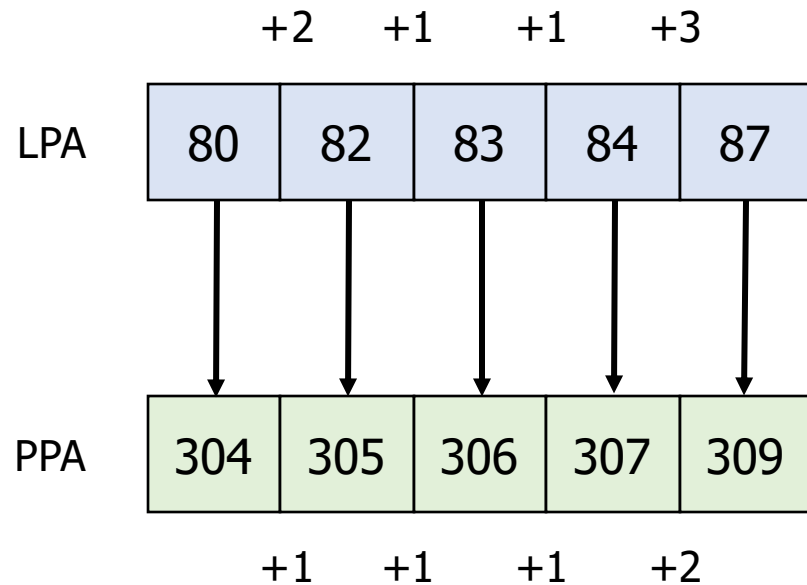
2.1.1. Learning Irregular Patterns

- Irregular Stride Access Pattern



2.1.1. Learning Irregular Patterns

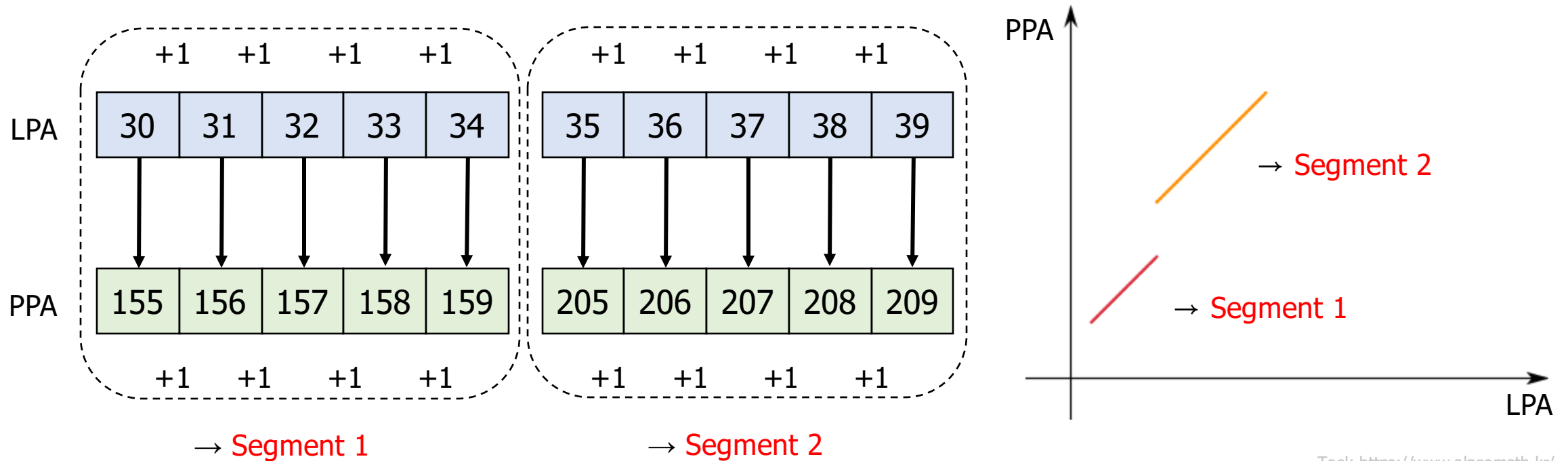
- **Relaxed Linear Regression** by Using Error Bound $[-\gamma, \gamma]$
 - If Address Mapping is **Within Pre-configured Error Bounds**
Then Allows **Forming a Line(=Segment)**



(b) Inaccurate Linear Approximation

2.1.2. Using Piecewise Linear Regression

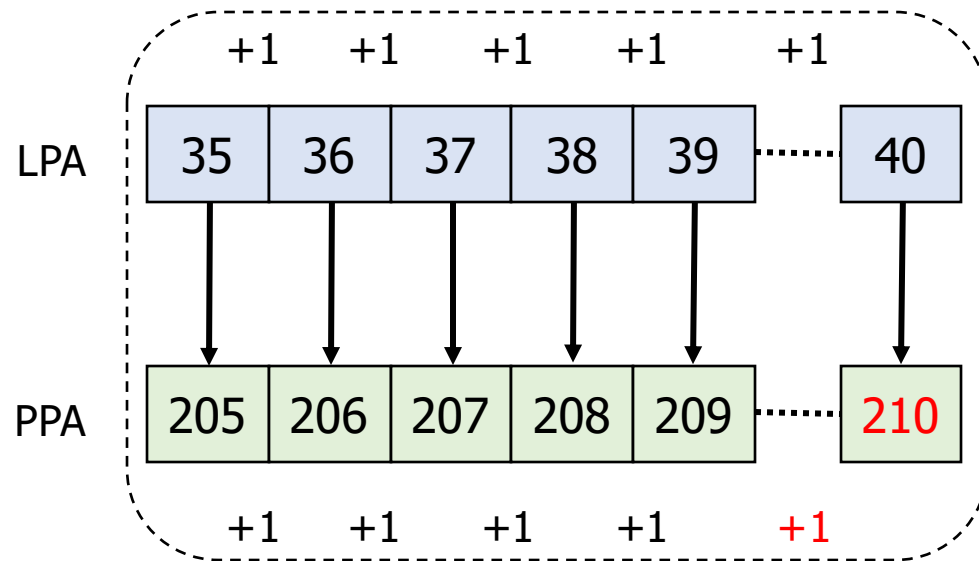
- Linear Regression but **Piecewise**
 - Splitting Address Space** into Several Segments
 - Each Segment Has **Own Linear Equation**



Tool: <https://www.algeomath.kr/>

2.1.2. Using Piecewise Linear Regression

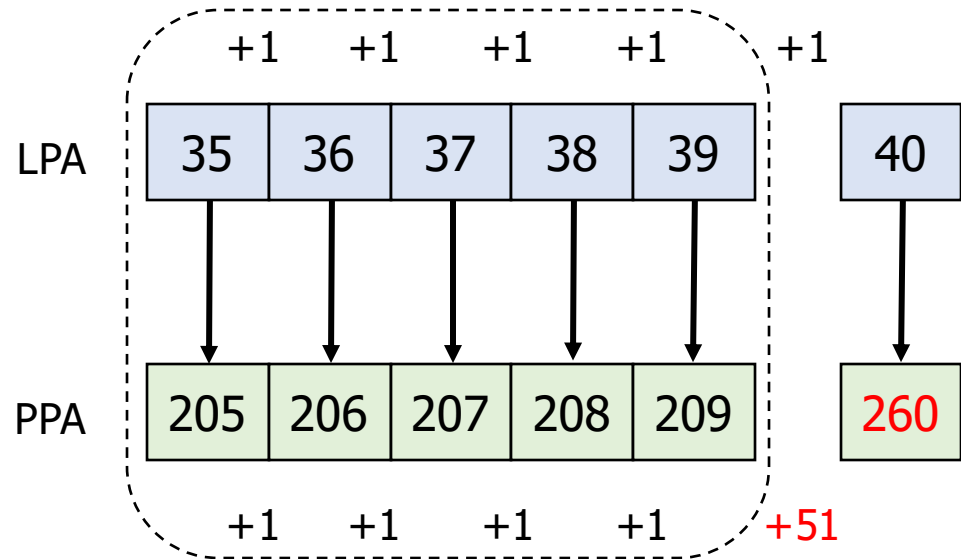
- Linear Regression but **Piecewise**
 - When New Address Mapping is Created
 - Case 1: **Within Error Bound** of Existing Segments



→ **Increase Length** of That Segment

2.1.2. Using Piecewise Linear Regression

- Linear Regression but **Piecewise**
 - When New Address Mapping is Created
 - Case 2: **Out of Error Bound** of Existing Segments



→ Be a **New Learned Segment**

2.1.3. Using Learned Index Segment

- Learned Index Segment
 - Searched for Calculating PPA Using LPA
 - Can Compact In-bound Mapping Entries into A Segment
 - Segment is Only 8-Byted Struct → ex. 20 Pages Can be Compacted to 8 Bytes!

- | 1 | 1 | 2 | 4 |
|-----------|-----|-----|-----|
| S_{LPA} | L | K | I |

(Byte)

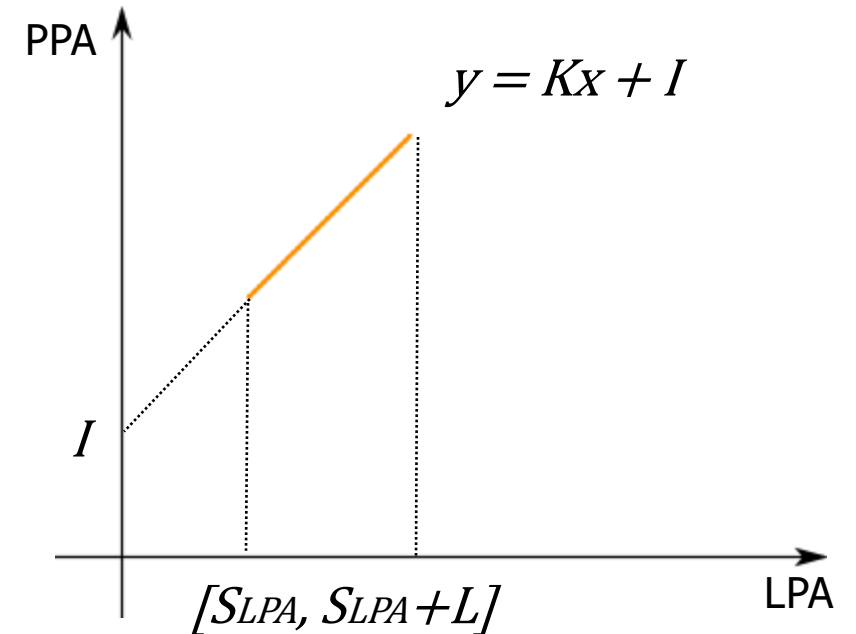
1	1	2	4
S_{LPA}	L	K	I

2.1.3. Using Learned Index Segment

- Components of Learned Index Segment

$SLPA$	L	K	I
--------	-----	-----	-----

- $SLPA$: **Start LPA Offset** of that Segment
- L : **Total Count**(Length) of LPA
that Belong to that Segment
(Avg=20.3, Max=256)
- K : **Slope** of Increasing Address Values
- I : **Base PPA** of that Segment



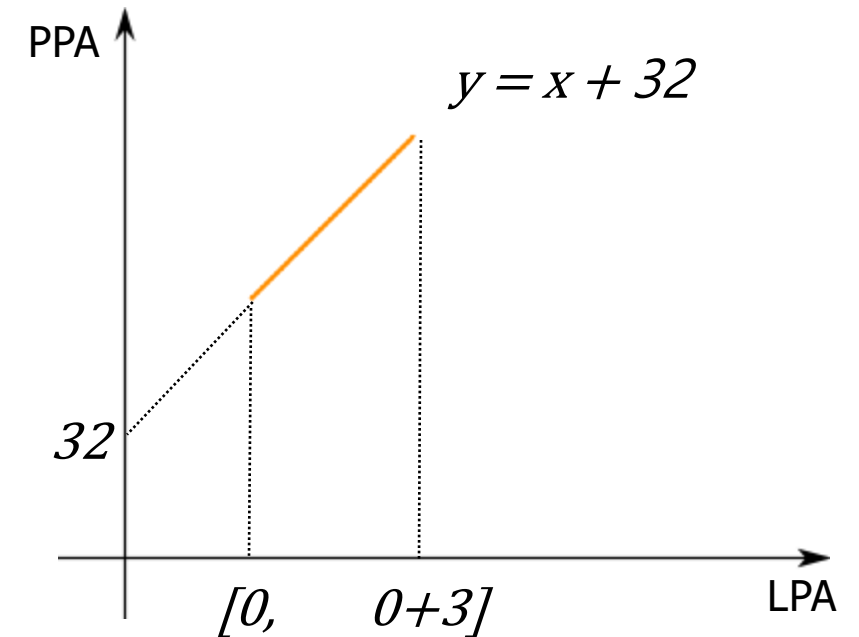
Tool: <https://www.algeomath.kr/>

2.1.3. Using Learned Index Segment

- Components of Learned Index Segment

$SLPA$	L	K	I
--------	-----	-----	-----

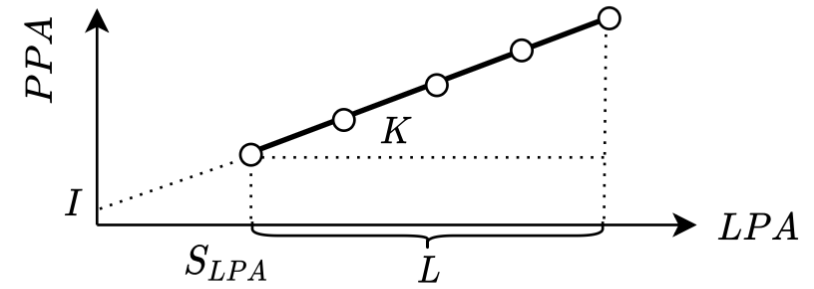
- $SLPA$: **Start LPA Offset** of that Segment
- L : **Total Count**(Length) of LPA that Belonging to that Segment (Avg=20.3, Max=256)
- K : **Slope** of Increasing Address Values
- I : **Base PPA** of that Segment



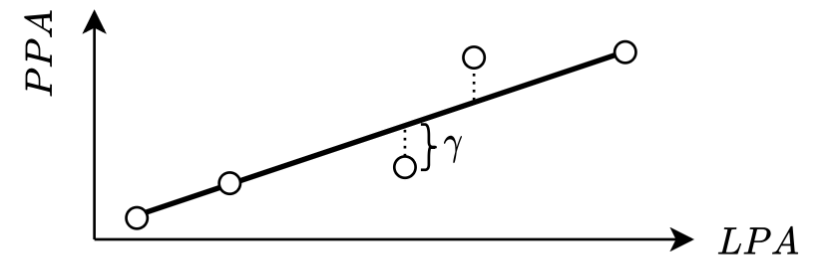
Type	LPAs	PPAs	Index Segment			
Accurate	[0, 1, 2, 3]	[32, 33, 34, 35]	0	3	1.00	32

2.1.4. 2 Types of Learned Segment

- Accurate Segment
 - $\text{Error Bound}(\gamma) = 0$
 - $\text{Calculated PPA} == \text{Actual PPA}$
- Approximate Segment
 - $\text{Error Bound}(\gamma) > 0$
 - **Calculated PPA Can be Different from Actual PPA**
→ Important Challenge



(a) Precise Linear Approximation



(b) Inaccurate Linear Approximation

2.1.5. Calculating PPA using LPA

- $PPA_{predicted} = \lceil K \times LPA + I \rceil$
 - $SLPA=0, L=3, K=1, I=32$ (Accurate)
 If $LPA = 2$, then $PPA_{predicted} = 34$
 → Correct
 - $SLPA=0, L=5, K=0.56, I=64$ (Approximate)
 If $LPA = 4$, then $PPA_{predicted} = 67$
 But **Actual PPA = 66**
 → **Address Misprediction**

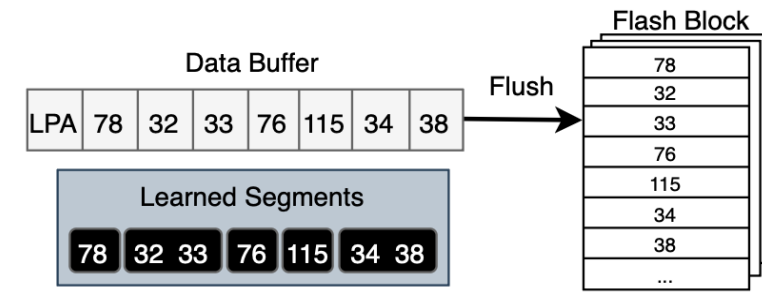
	1B	1B	2B	4B
Segment	S_{LPA}	L	K	I

Type	LPAs	PPAs	Index Segment			
Accurate	[0, 1, 2, 3]	[32, 33, 34, 35]	0	3	1.00	32
Approximate	[0, 1, 4, 5]	[64, 65, 66, 67]	0	5	0.56	64

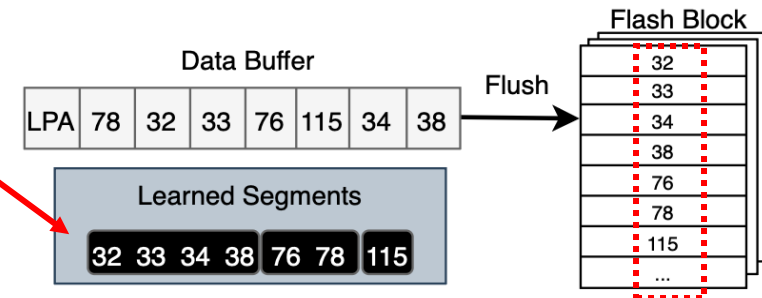
2.2. To Improve Learning Efficiency

- Sorts Flash Pages in Data Buffer in SSD Controller

- Before Write to Flash Chip
(= Allocates PPA Sequentially)
- Based on LPA
 - Forms Simple Mapping Patterns between LPA and PPA
 - Length of Segments Can Increase



(a) Unoptimized learned segments



(b) Optimized learned segments with sorting

2.3. Workflow of Creating Segments

1) Collecting Data

- Tiny Write Requests are Accumulated in Data Buffer
- Till the **Buffer is Full**

2) Sorting Pages in Data Buffer

- LeaFTL **Gets LPA & PPA** (to be Allocated) without Collecting Overhead
- **Sorts Pages** by LPA to Form Simple Mapping Patterns

3) Creating Learned Segments

- **Learns Mapping Patterns** Using Piecewise Linear Regression
- **Creates Learned Segments** Based on Error Bound
- Flush Occurs Finally

3. How to Manage Learned Segments?

3.1. Overcoming Out-of-place Update

- Out-of-place Update of Data in SSD
 - If Host Modifies Old Data, SSD Updates Data to New PPA
 - Then **Learned Segments Can Become Inaccurate**
- Problems of In-place Update of Learned Segments
 - Should Read All of LPAs in that Segment → Additional Flash Access Occurs
 - Linearity Breaks Down → Segment Fragmentation Occurs

3.1.1. Using Log-Structured Mapping Table

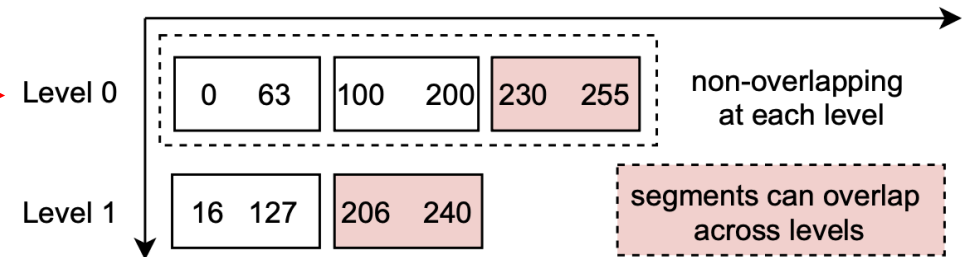
- Multi-level Log Structure by **Temporal Order**

- Recently** Learned Segments

- Append to **Top-level**

- Old** Segments

- **Descend** to Lower Level



- Segment **Overlap**

- Several Segments that **Indicate Overlapping LPA Range**

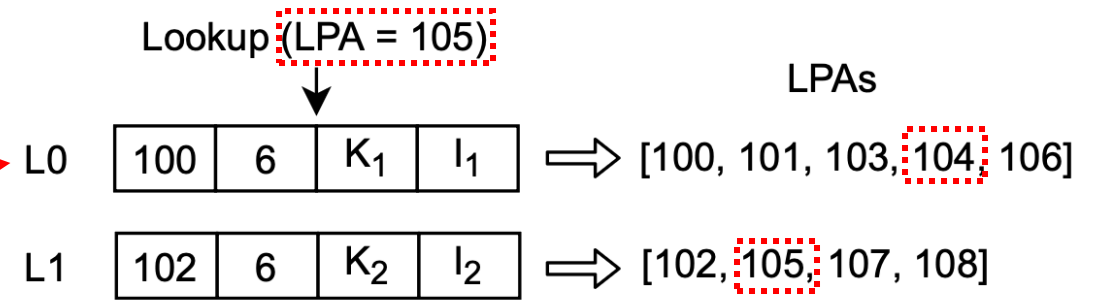
- In Same Level: Unable

- Across the Level: Able**

3.1.2. Problem of LSMT Overlapping

- If Two Approximate Segments Have **Overlapped LPA Ranges?**

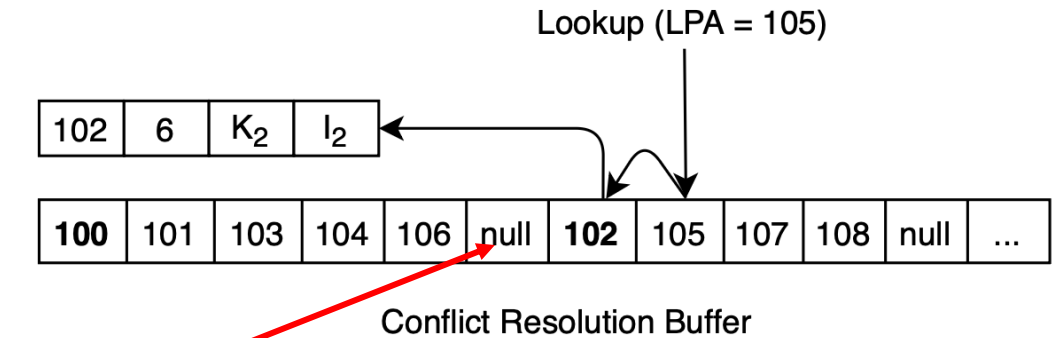
- Approximate Segment
Can't Lookup Target LPA Accurately
- Possibility that **Returns Wrong Segment** → Returns Wrong PPA



(a) Approximate index segments that index overlapped LPAs.

3.1.3. Using Conflict Resolution Buffer

- CRB(Conflict Resolution Buffer)
 - Stores that **LPAs of Approximate Segments**
 - Appended when **Approx. Segments Creates**
 - Searched when **LPA Lookup of Approx. Segment** To Get Accurate LPA
 - Each Segment is Sorted Based on **Start LPA**
 - Each Segment is Separated to **null byte**



(b) Resolve the conflict between approximate segments with CRB

3.2. Workflow of Managing Segments

- Insert/Update of Learned Segments
- Searching LPA
- Segments Compaction

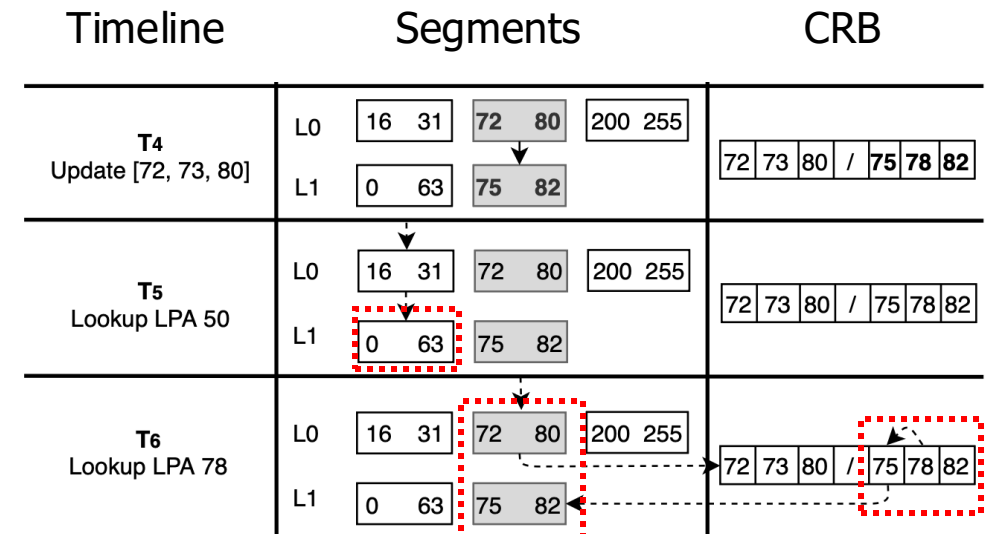
3.2.1. Insert/Update of Learned Segments

- $T_1 \rightarrow T_2$
 - Overlapped Old Segment [0, 63] is Descended to L1
- $T_2 \rightarrow T_3 \rightarrow T_4$
 - New LPAs of Approx. Segment is Updated to CRB
 - If T_3 was "Update [72, 73, 78]" ?
 - LPA 72, 73 of [72, 73, 78] Get Unmapped
 - Segment Becomes [78] in L1 & 78 Only Remains in CRB

Accurate Segment		Start	End	Approximate Segment		Start	End
Timeline	Segments				CRB		
T_0 Initial Snapshot	L0	0	63				
T_1 Update LPAs 200 - 255	L0	0	63	200	255		
T_2 Update LPAs 16 - 31	L0	16	31	200	255		
	L1	0	63				
T_3 Update [75, 78, 82]	L0	16	31	75	82	200	255
	L1	0	63				75 78 82
T_4 Update [72, 73, 80]	L0	16	31	72	80	200	255
	L1	0	63	75	82		72 73 80 / 75 78 82

3.2.2. Searching LPA

- T4 → T5
 - Finds L1 if Target LPA(50) is NOT in L0
- T5 → T6
 - Firstly Searches [72, 73, 80] in CRB
Because [72, 73, 80] is Approx. Segment
 - **Next Searches [75, 78, 82] in CRB**
Because [75, 78, 82] is Approx. Segment
 - Finally Finds LPA 78



3.2.3. Segments Compaction

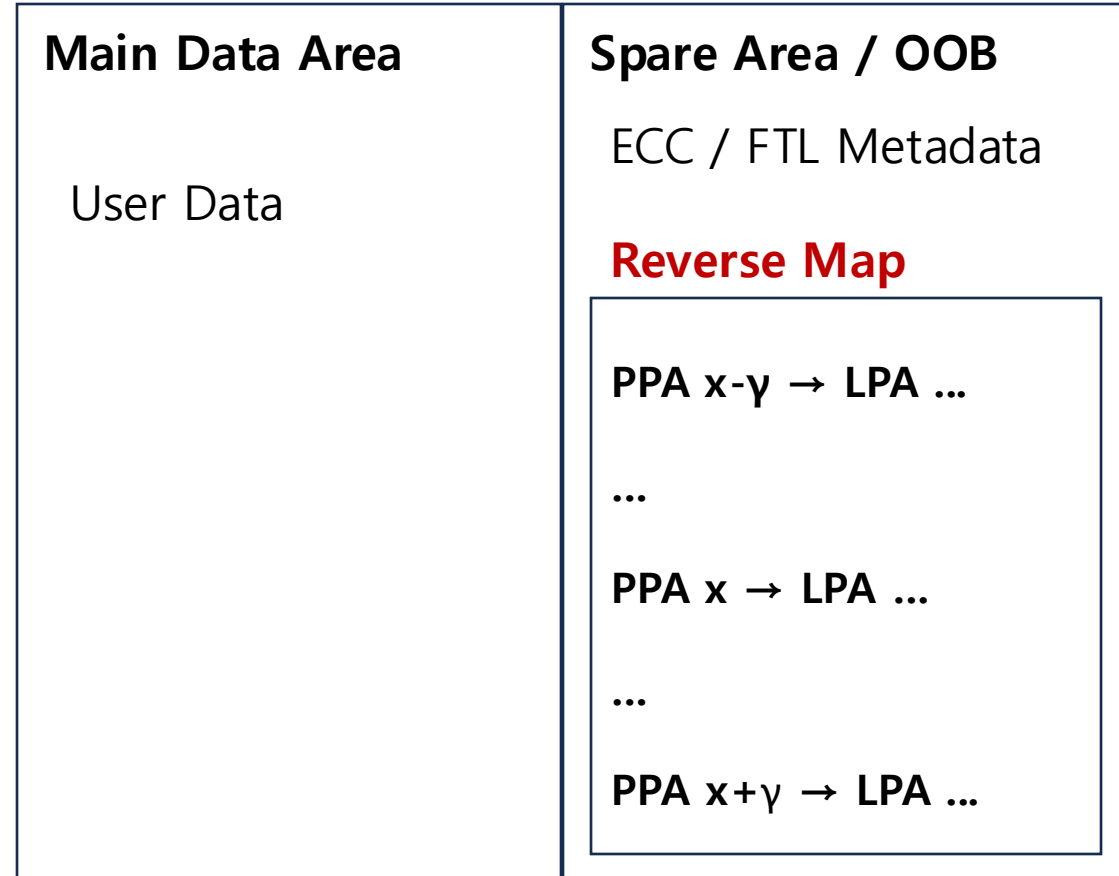
- T7 → T8
 - Put All of Segments in L0 into L1
 - Old LPAs are Unmapped
& Segment that No Valid LPAs are Deleted
 - Then Original L0 is Empty
 - Delete that Level & Invalid CRB
 - Original L1 Becomes New L0
 - Memory Space Can be Optimized by Compaction

Timeline	Segments				CRB
T7 Update LPAs 32 - 90	L0	16 31	32 90	200 255	75 78 82
	L1	0 63	75 82		
T8 Compaction	L0	0 15	16 31	32 90 200 255	

3.3. Misprediction Handling

- Handle Address Misprediction
- OOB(Out-of-Band)

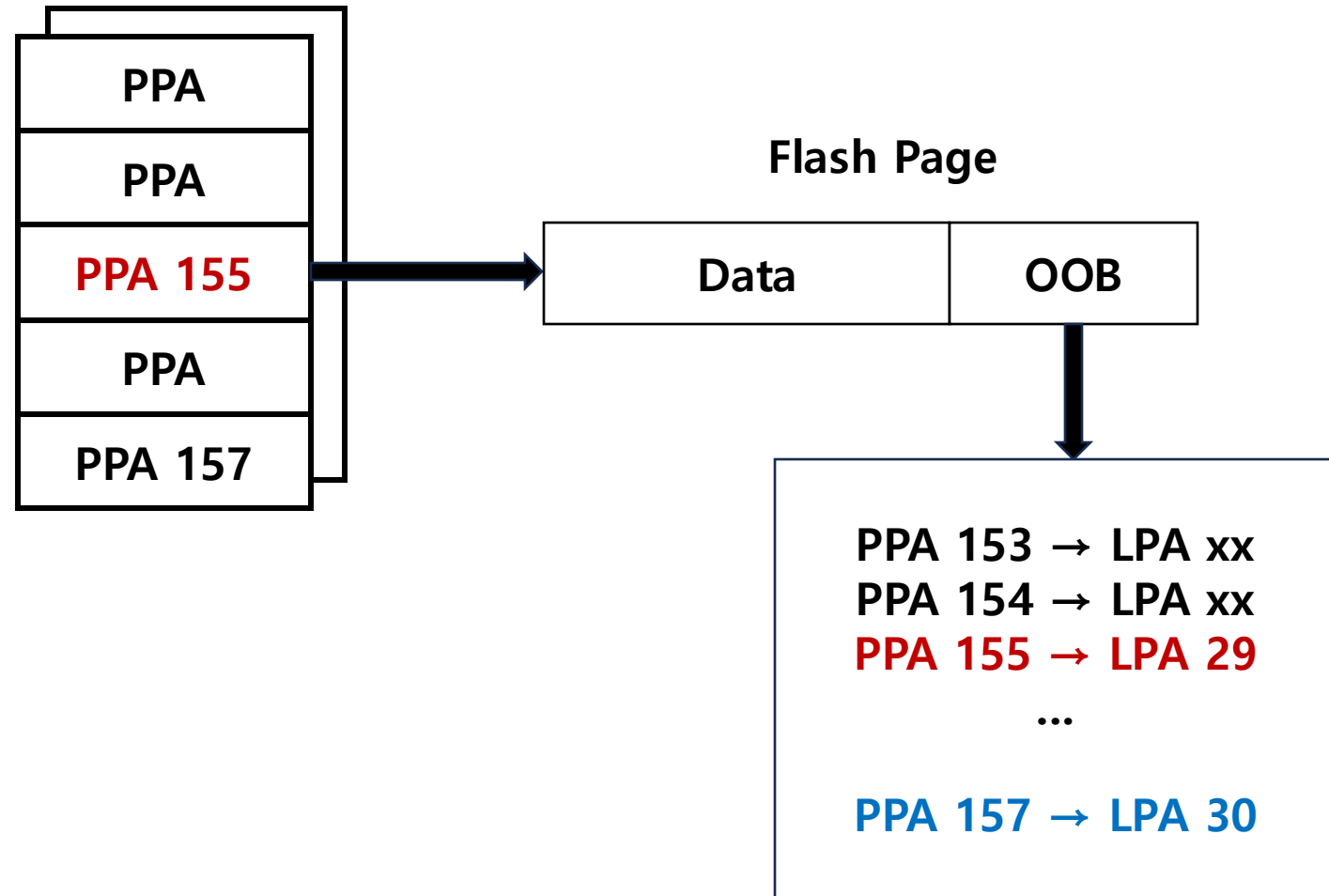
NAND Page



3.3. Misprediction Handling

■ Example

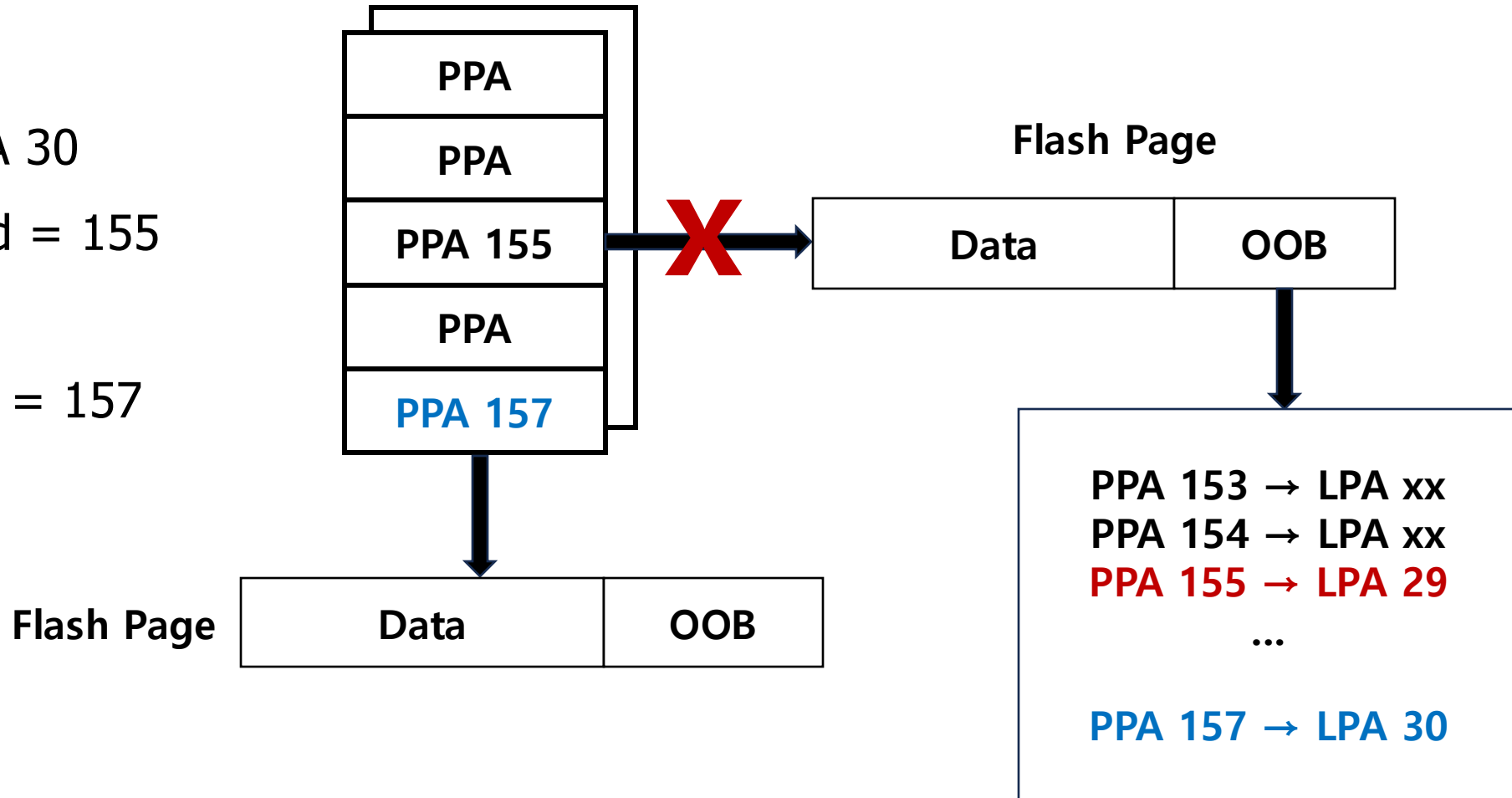
- Read LPA 30
- PPA_pred = 155
- $\gamma = 2$
- PPA_real = 157



3.3. Misprediction Handling

■ Example

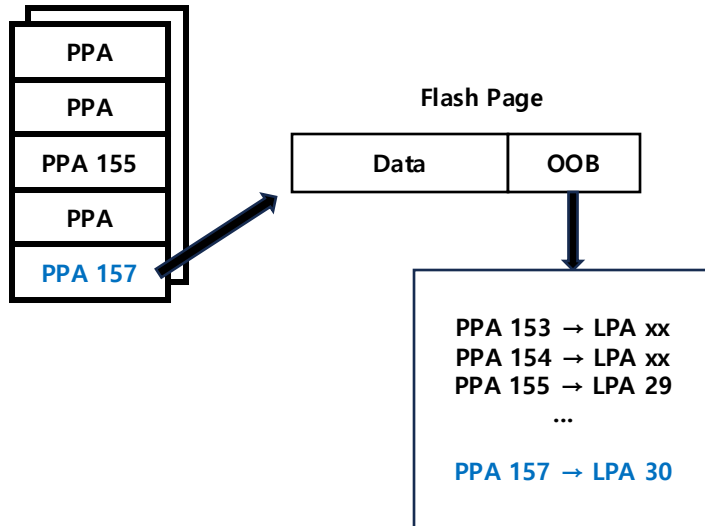
- Read LPA 30
- PPA_pred = 155
- $\gamma = 2$
- PPA_real = 157



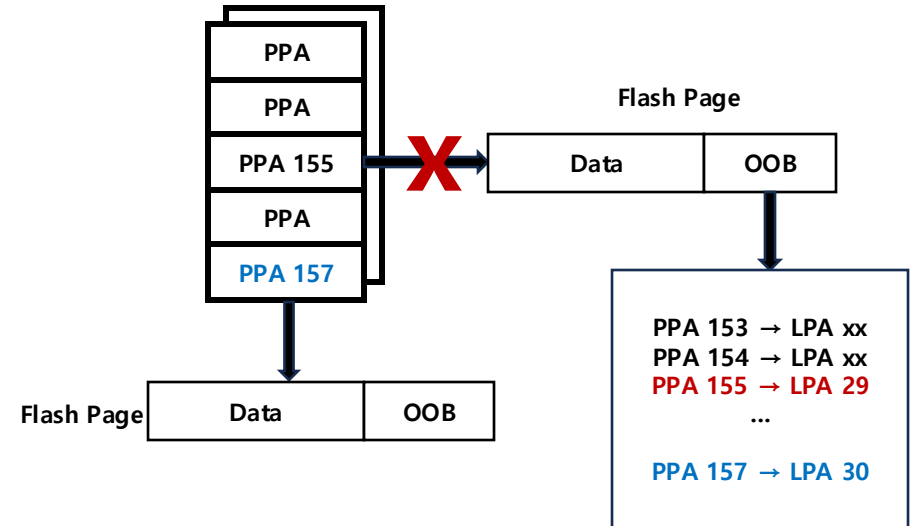
3.3. Misprediction Handling

- $PPA_real \in [PPA_pred - \gamma, PPA_pred + \gamma]$

Correct prediction: 1 flash read



Misprediction: 2 reads total

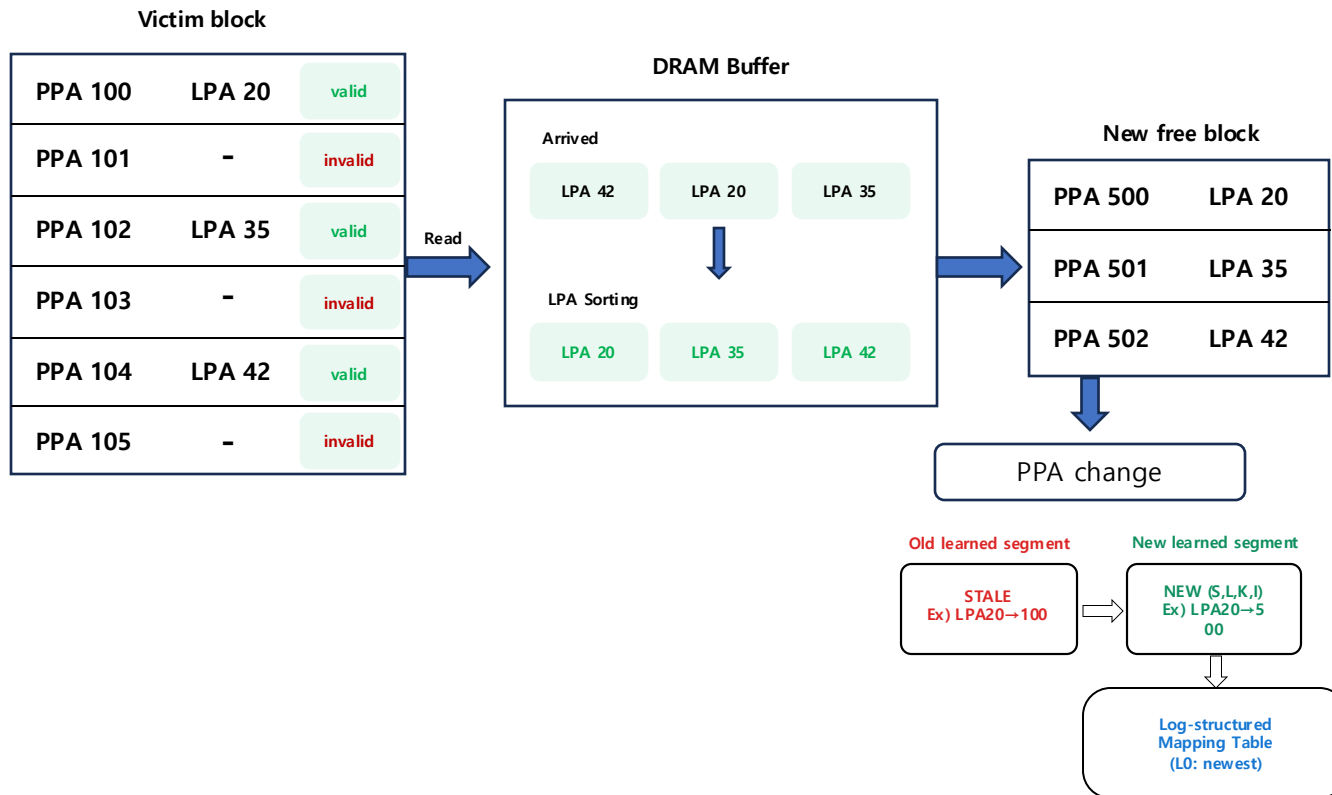


Penalty = exactly one additional flash read

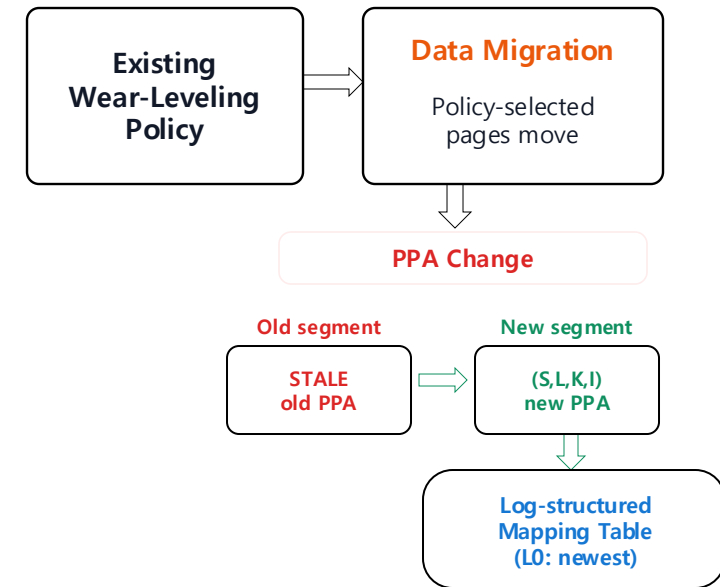
4. Preserve Other FTL Functions

■ GC / Wear-Leveling

GC



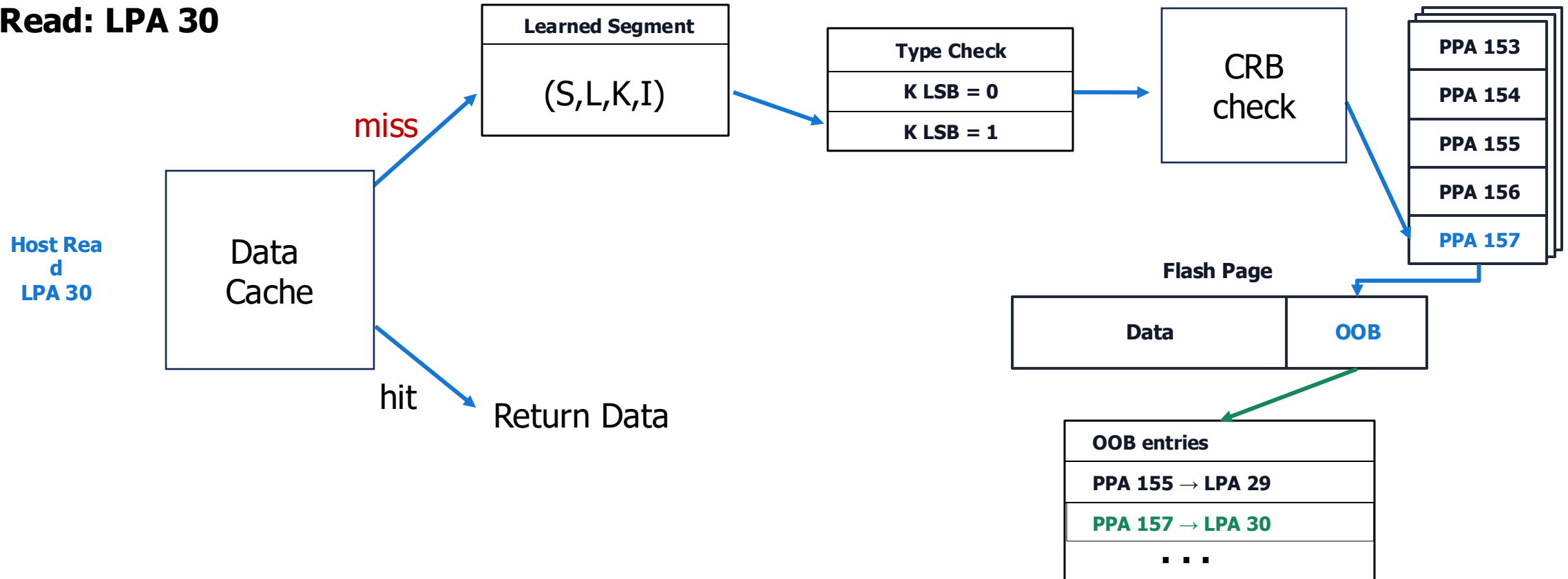
Wear-Leveling



5. I/O operation

- READ/WRITE

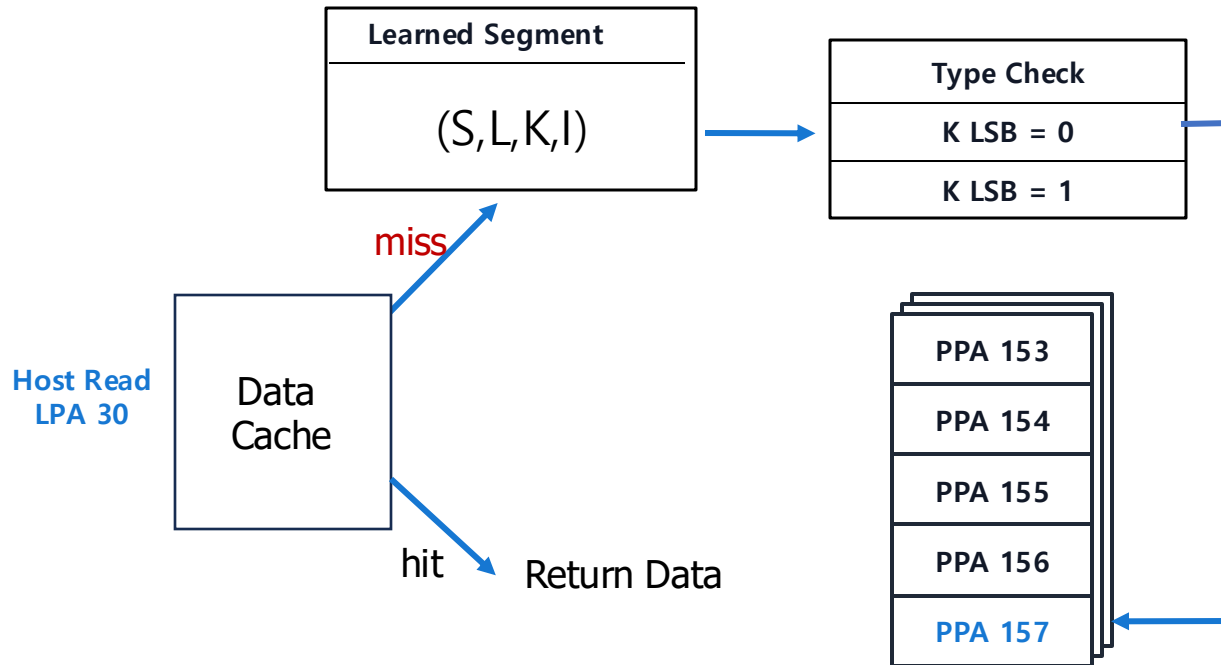
Read: LPA 30



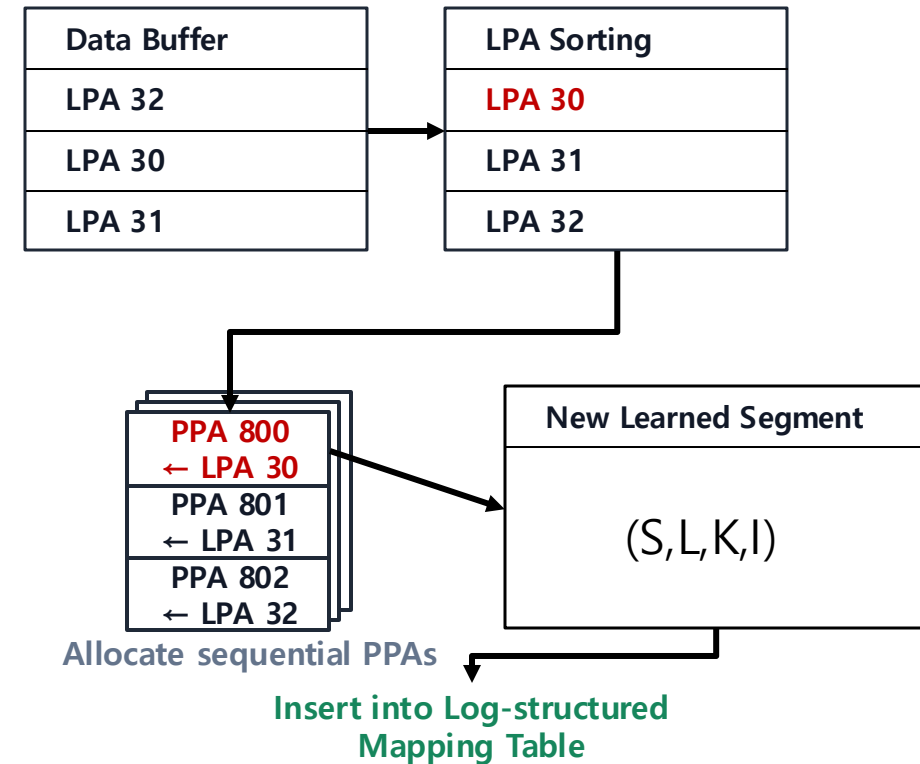
5. I/O operation

■ READ/WRITE

Read: LPA 30



Write: Buffer Flush



6. Evaluation

■ Setup

SSD Simulator

wiscSim

2 TB capacity · 4 KB page
1 GB DRAM · 16 channels · OOB 128 B

Real SSD Prototype

1 TB capacity · 16 KB page

16 channels · 256 pages/block

■ Workload

SSD Simulator

MSR · enterprise servers
FIU · university servers

Real SSD Prototype

Data-Intensive Applications

FileBench · BenchBase

■ Baselines

- DFTL, SFTL

6. Evaluation

■ Mapping Table Memory Reduction

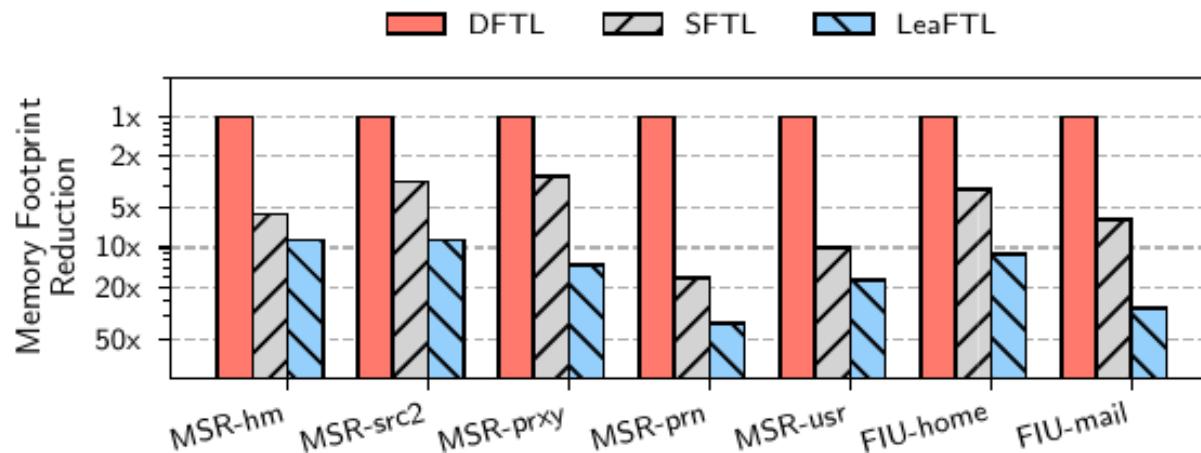


Figure 15 · Mapping-table footprint reduction

Reduction factor ↑ = smaller mapping table

2.9x

smaller on average
vs. SFTL

up to 5.3x

Core objective: reduce the
mapping-memory footprint

More mappings represented by fewer learned segments → **More free SSD DRAM**

6. Evaluation

■ Performance

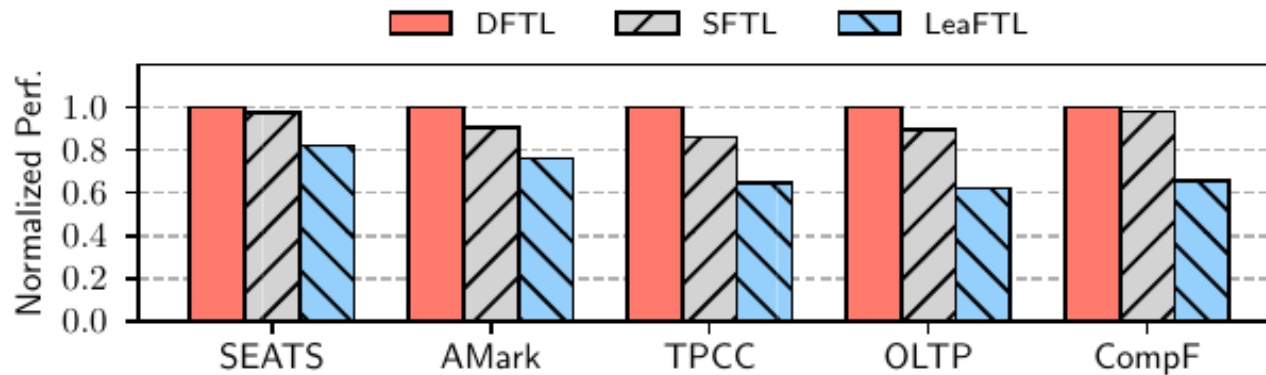


Figure 17 · Real SSD prototype(lower is better)

1.4×
average speedup

1.5×
maximum speedup

vs. DFTL and SFTL · real SSD workloads

Mapping table ↓ → More DRAM for Data Cache → Cache hit ↑ → Flash access ↓ → Performance ↑

6. Evaluation

■ Sensitivity Analysis of Error Bound γ

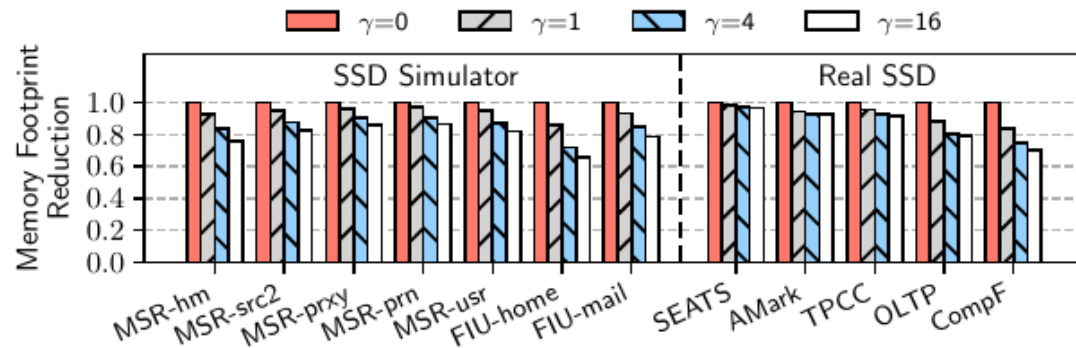


Figure 19 · Mapping-table footprint (lower is better)

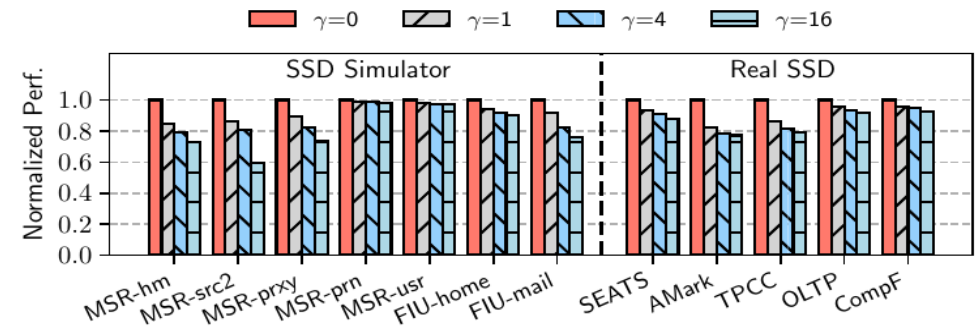


Figure 21: Performance with various γ (lower is better).

Figure 21 · Normalized performance (lower is better)

Larger γ → More mappings per segment → Smaller mapping table → More DRAM for data cache → Better performance

$\gamma = 16$ vs. $\gamma = 0$ · Mapping table 1.3× smaller · Performance 1.3× faster

7. Conclusion

- LeaFTL uses a simple but effective learning-based technique to reduce memory consumption
- LeaFTL stores learned segments in a log-structured manner to avoid re-learning
- LeaFTL uses OOB metadata to verify its address translation
- LeaFTL consumes 2.9× less memory and improves performance by 1.4×

Thank you

Appendix

■ Compression vs. Misprediction + Overhead

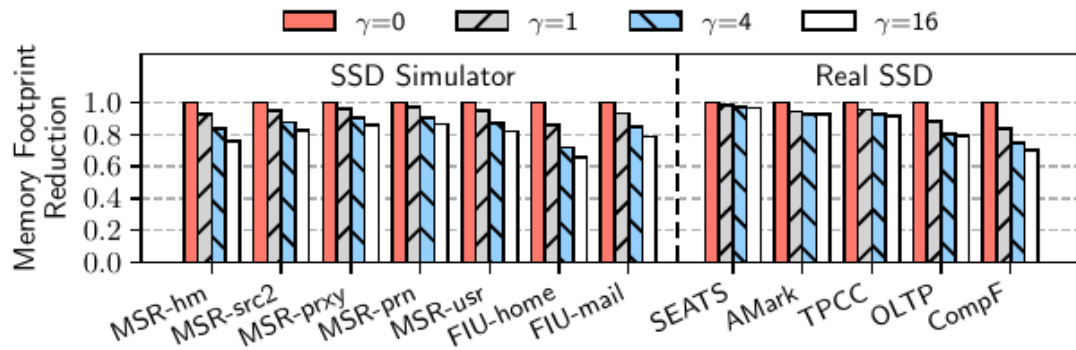


Figure 19 · Footprint ↓ as γ increases

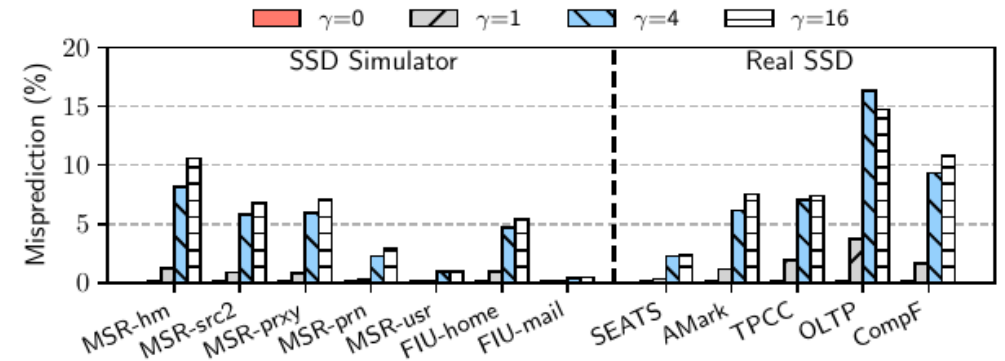


Figure 24 · Misprediction ↑ as γ increases

Larger γ → better compression · more mispredictions

+1 flash read

OOB recovery penalty